AgentSouriva

Technical Documentation and Development Log

9 October 2026

Contents

1 Project Overview **2**

2 System Architecture and Document Flow **3**

3 Features and Components **4**

4 API and MCP Interfaces **6**

5 Models and Runtime Configuration **9**

6 Evaluation, Tracking, and Model Adaptation **10**

7 Project Build Evolution **12**

1 Project Overview

AgentSouriva is a local, evidence-grounded agent for reasoning over PDF, Markdown, and text collections.

It is built around a simple question: **how can an AI agent answer from documents while keeping its evidence, tool use, and uncertainty visible?**

Given a collection of documents, AgentSouriva indexes their contents, retrieves relevant evidence for each question, and returns source-aware answers. It can validate citations against retrieved evidence, route approved tasks through dedicated tools, and abstain when the available documents do not sufficiently support an answer.

The project is designed as a compact environment for exploring **trustworthy document-grounded agents**, including retrieval, constrained tool use, citation validation, abstention, evaluation, and optional model adaptation.

1.1 Why AgentSouriva?

Retrieval-augmented systems can make language models useful over private or specialized documents, but retrieval alone does not guarantee that an answer is supported by the retrieved evidence.

AgentSouriva makes the important parts of this process explicit:

- **Evidence retrieval**: answers are grounded in chunks retrieved from the indexed corpus.
- **Source-aware citations**: responses retain references to the evidence used.
- **Citation validation**: citation IDs are checked against the evidence actually retrieved.
- **Controlled tool use**: the agent operates through a small set of approved tools.
- **Abstention**: unsupported document questions can return an abstention rather than an unsupported answer.
- **Evaluation**: retrieval, citation behavior, abstention, tool use, unsupported answers, and latency can be measured directly.

This makes AgentSouriva useful both as a local document assistant and as an experimental platform for studying grounded agent behavior.

1.2 Example Uses

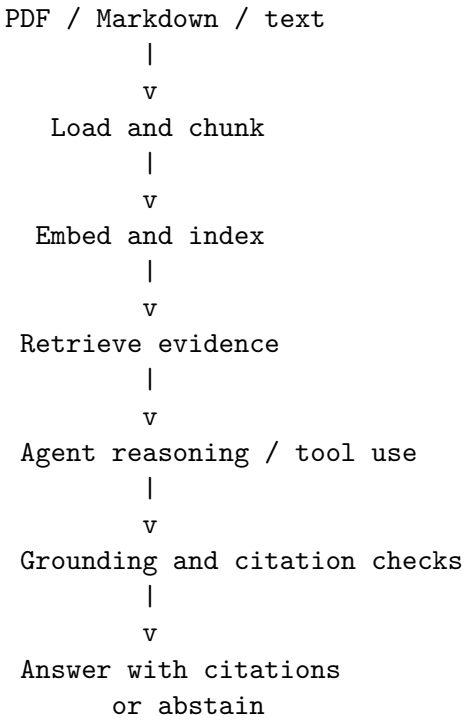
AgentSouriva can be used to:

- compare methods, datasets, metrics, and reported results across papers;
- answer questions from technical manuals and architecture documentation;
- extract authentication, endpoint, and usage details from API documentation;
- compare specifications across documents with source references;
- retrieve documented quantities and perform approved calculations;
- identify when the indexed documents do not contain enough information to answer a question.

1.3 License

AgentSouriva is available under the MIT License.

2 System Architecture and Document Flow



2.1 Build the document index

Documents are loaded and divided into chunks. Each chunk retains source information so retrieved evidence can later be traced back to its document and location.

The chunks are embedded and stored in a FAISS index for retrieval.

2.2 Retrieve relevant evidence

When a question arrives, AgentSouriva searches the index for the document chunks most relevant to that question.

Retrieval can also be called independently when an application needs the evidence itself rather than a generated answer.

2.3 Route the request

The agent decides whether the request should use document retrieval or an approved tool.

Document questions use retrieved evidence. Approved arithmetic is routed through a deterministic calculator rather than delegated to the language model.

2.4 Generate a grounded answer

For document questions, the model receives the question together with the retrieved evidence.

Retrieved document text is treated as untrusted data in the prompt rather than as instructions for the agent.

2.5 Validate the result

Before returning the response, AgentSouriva checks that citation IDs in the generated answer correspond to evidence retrieved above the configured relevance threshold.

The result is either a supported answer with citations or an abstention when the indexed evidence is insufficient.

2.6 Document Ingestion Semantics

The `/documents` interface accepts paths to source files and builds the retrieval index and citation metadata for the selected files. Loading another set of documents replaces the previously indexed set.

Supported source extensions are `.pdf`, `.md`, `.markdown`, and `.txt`. For PDFs, citation locations use page numbers. Markdown and text sources retain document and chunk locations.

Documents are normalized into source-aware chunks before embedding and indexing. The document corpus can be replaced by a subsequent ingestion request, rather than cumulatively appended.

3 Features and Components

3.1 Local document retrieval

Document chunks are indexed locally using FAISS.

The default `HashEmbeddings` backend uses deterministic feature hashing and requires no downloaded embedding weights. It favors lexical overlap and provides a lightweight replacement point for learned embedding backends.

3.2 Grounded generation

Retrieved document chunks are supplied as evidence for the model rather than treated as trusted instructions.

Generated citation IDs are validated against the retrieved evidence before the answer is returned.

3.3 Abstention

When the available evidence does not sufficiently support a document answer, the system can abstain rather than return an unsupported answer.

3.4 Constrained tools

Agent actions are limited to explicitly approved tools. Retrieval and arithmetic remain separate from general model generation.

3.5 Local-first model boundary

The default model and embedding backends can operate without a separately hosted model server.

For larger local models, AgentSouriva exposes an OpenAI-compatible model boundary. GPT-OSS-20B can be connected through any compatible serving setup without coupling the retrieval and grounding layers to a particular inference engine.

3.6 Observable agent runs

Optional MLflow integration can record questions, retrieved evidence, scores, answers, citations, tool use, action traces, latency, and relevant configuration.

3.7 Evaluation

The repository includes direct evaluation for:

- retrieval Recall@k source hits;
- citation correctness;
- unsupported-answer rate;
- abstention accuracy;
- tool-use accuracy;
- mean latency.

Optional RAGAS evaluation adds:

- faithfulness;
- answer relevance;
- context precision;
- context recall.

3.8 Approved Tool Surface

AgentSouriva keeps tool use explicit and restricted. The current tool set includes:

- Document search;
- Retrieval of a specific document chunk;
- Deterministic arithmetic.

Approved arithmetic is routed to the deterministic calculator instead of relying on model-generated arithmetic.

3.9 Repository Structure

```
app.py                    FastAPI interface

souriva/
|-- agent.py              action routing and orchestration
|-- rag.py                loading, chunking, embeddings, FAISS, retrieval
|-- model.py              model adapters
|-- alignment.py          citation validation and abstention
|-- tools.py              approved document tools and calculator
|-- mcp_server.py         MCP interface
|-- tracking.py           MLflow run and evaluation logging
`-- evaluation.py         direct quality metrics

benchmarks/
`-- ragas_eval.py         evaluation CLI and RAGAS integration
```

```
training/                optional model adaptation

docs-tex/build-logs/     per-change engineering reports (LaTeX)
```

4 API and MCP Interfaces

AgentSouriva exposes a small FastAPI interface. `GET` is used to read status information, while the `POST` endpoints accept request data such as document paths, retrieval queries, or questions.

Method	Endpoint	Purpose
`GET`	`/health`	Check application, index, and model-backend status
`POST`	`/documents`	Load a new set of documents for indexing
`POST`	`/retrieve`	Retrieve ranked document evidence for a query
`POST`	`/ask`	Ask a grounded question or invoke an approved action

4.1 Document Ingestion Interface

All endpoints belong to the AgentSouriva FastAPI server. A separate HTTP client—for example, an administration script, command-line client, web front end, test harness, or another service—sends requests to that server. The client does not call the retrieval store or model directly. An operator or administration process calls `/documents` when establishing or refreshing the searchable document collection. After that collection is ready, a user-facing application calls `/ask` when it needs a complete grounded answer, or `/retrieve` when it needs ranked evidence without answer generation. A deployment monitor can call `/health` at any time to check readiness.

For example, an administrator can first use a command-line HTTP client to send document paths using `POST /documents`. Later, when a user submits a question in a web interface, that web interface acts as the HTTP client and sends the question using `POST /ask`.

Aftab: RESOLVED– here give code listings of actually how are teh calls made in the code, I think you already have them implemented in the source

The repository's API tests exercise this client-to-server path with `httpx`. The following excerpt from `tests/test_api_documents.py` shows the reusable in-process HTTP client and the two calls. A deployed web or command-line client sends the same methods, paths, and JSON bodies over the network rather than through `ASGITransport`.

```
def request(app, method, path, json):
    async def send():
        transport = httpx.ASGITransport(app=app)
        async with httpx.AsyncClient(
            transport=transport, base_url="http://test"
        ) as client:
            return await client.request(method, path, json=json)
    return asyncio.run(send())

response = request(
    app, "POST", "/documents",
    {"paths": [str(document)]},
```

```
)
answer = request(
    app, "POST", "/ask",
    {"question": "How does token authentication work?"},
)
```

On the server side, the corresponding FastAPI route declarations in `app.py` deserialize those JSON bodies into typed request objects and invoke the ingestion or agent flow. The excerpt below omits error handling and response-field assembly while retaining the implemented call path.

```
@app.post("/documents", response_model=IngestResponse)
async def ingest_documents(request: DocumentsRequest):
    count = ingest_paths(runtime.store, request.paths, ...)
    return IngestResponse(chunks=count, sources=...)

@app.post("/ask", response_model=AskResponse)
async def ask(request: AskRequest):
    result = runtime.agent.ask(request.question)
    return AskResponse(answer=result.answer.text, ...)
```

AgentSouriva retrieves evidence, invokes the configured model boundary when needed, validates the result, and returns JSON to the web interface for presentation to the user.

The `/documents` endpoint accepts a JSON object containing file paths and builds the index and source-aware citation metadata for those files. Ingestion is replacement-based rather than append-based: each successful call makes the submitted files the complete active corpus and replaces the previously indexed chunks. For example, if one request indexes `architecture.md` and `api.txt`, and a later request submits only `handbook.pdf`, subsequent `/retrieve` and `/ask` requests search `handbook.pdf` rather than the earlier two files. This later ingestion is useful when an operator publishes a revised or entirely different collection. Request bodies are transmitted as `application/json`.

A representative request-body structure from the project documentation is:

```
{"paths": ["data/sample_docs/architecture.md", "data/sample_docs/api.txt"]}
```

4.2 Grounded Question Interface

The `/ask` endpoint accepts a JSON question, initiates the answer flow, and returns the answer together with structured citation information, an abstention indicator, a reason, and an action trace.

The request-body structure is:

```
{"question": "What vector index does AgentSouriva use?"}
```

A grounded response includes both the answer and structured citation information:

```
{
  "answer": "AgentSouriva loads local documents ... FAISS index. [chunk-...]",
  "citations": [
    {
```

```
      "id": "chunk-...",
      "source": "architecture.md",
      "location": "document, chunk 1",
      "score": 0.4
    }
  ],
  "abstained": false,
  "reason": null,
  "actions": ["RETRIEVE", "ANSWER"]
}
```

The structured response exposes both the answer and the evidence trail to applications built on top of AgentSouriva.

4.3 Direct Retrieval Interface

The `/retrieve` endpoint returns ranked document chunks along with source metadata and relevance scores, without first generating an answer. Its JSON request includes a query and an optional retrieval limit, as illustrated by the source request-body structure:

```
{"query": "citation policy", "top_k": 4}
```

4.4 Approved Tool Routing

The `/ask` endpoint can route approved arithmetic requests to the deterministic calculator rather than delegate calculation to the language model. The approved tool surface consists of document search, chunk retrieval, and deterministic arithmetic.

4.5 Model Context Protocol (MCP)

AgentSouriva can expose its approved capabilities through the Model Context Protocol, allowing an MCP-compatible client to call its document and calculator tools.

The stdio MCP interface exposes the following tool signatures:

```
search_documents(query, top_k=4)
get_document_chunk(chunk_id)
calculate(expression)
```

The MCP interface uses the same persisted retrieval index and runtime configuration as the main API. It is an interface to the existing capabilities, not a separate retrieval implementation.

MCP-based interaction. An MCP-compatible host, such as an IDE assistant or another agent runtime, acts as the MCP client. It starts or connects to the AgentSouriva MCP server over standard input and output, discovers the `search_documents` tool, and invokes it with a query. The AgentSouriva MCP server receives that tool call and delegates it to `DocumentTools`.

`DocumentTools` is the transport-independent approved-tool wrapper defined in `souriva/tools.py`.

DocumentTools, defined in souriva/tools.py, is a wrapper groups the small set of operations that are explicitly allowed to be invoked – Its methods contain no HTTP- or MCP-specific code,

so different interfaces can reuse the same implementation. It holds the shared `FaissStore` and exposes three bounded operations: searching indexed documents, retrieving one chunk by its stable identifier, and evaluating restricted arithmetic. The MCP functions in `souriva/mcp_server.py` are thin protocol adapters that call these methods; `DocumentTools` performs the reusable application operation without depending on MCP itself. For a search call, it queries the shared FAISS index and converts the resulting evidence objects into dictionaries before the MCP server sends them back to the client. The communicating entities are therefore the external MCP host and the AgentSouriva MCP server; the user interacts with the host rather than issuing the tool call directly.

Non-MCP HTTP interaction. A web front end or command-line client can instead send `POST /retrieve` with a JSON query directly to the AgentSouriva FastAPI server. FastAPI calls the same underlying retrieval store and returns ranked evidence as an HTTP JSON response. The communicating entities are the HTTP client and the AgentSouriva FastAPI server, and no MCP host, tool discovery, or MCP transport participates. This form is appropriate when an application already integrates with AgentSouriva as an HTTP service; MCP is appropriate when an agent host needs AgentSouriva capabilities exposed as discoverable tools.

For example, a document-search web portal with a fixed backend contract can call `/documents`, `/retrieve`, and `/ask` directly: its developers already know which endpoint to call, and its user interface controls the request sequence. Because the portal already knows which HTTP endpoints to call, adding MCP would increase complexity without providing any useful new functionality. By contrast, an IDE assistant that already supports MCP can be configured to launch the AgentSouriva MCP server. The assistant discovers `search_documents`, `get_document_chunk`, and `calculate` at runtime and chooses among them while handling a user's request, without requiring custom HTTP integration code for each operation. In that situation, tool discovery and standardized invocation are the reason to use MCP.

5 Models and Runtime Configuration

5.1 Local and External Model Boundaries

The default model and embedding backends can operate without a separately hosted model server. For larger local models, AgentSouriva exposes an OpenAI-compatible model boundary. GPT-OSS-20B can be connected through any compatible serving setup without coupling the retrieval and grounding layers to a particular inference engine.

5.2 GPT-OSS-20B

AgentSouriva can connect to GPT-OSS-20B through an OpenAI-compatible model server. The configured model server must implement `POST /v1/chat/completions`.

In AgentSouriva, the adapter implements the `ModelAdapter` Python protocol, which provides a common interface between the agent's internal question-and-evidence representation and different answer-generation backends. The `OpenAICompatibleModel` adapter translates questions and retrieved evidence into OpenAI-compatible chat-completions API requests, sends them to the configured model server, and converts the responses, including generated text and referenced chunk identifiers, into AgentSouriva's `ModelReply` structure. The local `MockModel` follows the same protocol but operates without a model server. By relying on this shared interface, the agent keeps its application logic, retrieval, and grounding mechanisms independent of the underlying model-serving implementation.

Without an external model server, the default mock backend provides deterministic extractive responses for local operation and testing.

5.3 Runtime Settings and Configuration Interface

Runtime settings use the `SOURIVA_` prefix. The application reads configuration during startup. The following values are shown as configuration examples in the source README, not asserted as deployment defaults.

Setting	Source README example	Role
`SOURIVA_DATA_DIR`	`./artifacts`	Persisted index and runtime artifacts
`SOURIVA_CHUNK_SIZE`	`900`	Chunk-size setting
`SOURIVA_CHUNK_OVERLAP`	`120`	Overlap between text chunks
`SOURIVA_RETRIEVAL_TOP_K`	`4`	Number of retrieved chunks
`SOURIVA_MIN_RELEVANCE`	`0.18`	Minimum evidence-relevance threshold
`SOURIVA_MODEL_BACKEND`	`openai-compatible`	Model-backend selector
`SOURIVA_MODEL_BASE_URL`	`http://localhost:8001/v1`	Model server API base URL
`SOURIVA_MODEL_NAME`	`openai/gpt-oss-20b`	Model identifier
`SOURIVA_MODEL_API_KEY`	`local`	API key supplied to the model endpoint

5.4 Runtime Environment

The source README specifies Python 3.11 or newer. Runtime, evaluation, tracking, MCP, and training capabilities have distinct dependency requirements, as detailed in the chronological build logs.

6 Evaluation, Tracking, and Model Adaptation

6.1 Direct Evaluation

The included evaluation suite covers document retrieval, citation behavior, abstention, and calculator routing. The default evaluation reports directly observed metrics without requiring an evaluator model.

The direct evaluation metrics are:

- Retrieval Recall@k source hits;
- Citation correctness;
- Unsupported-answer rate;
- Abstention accuracy;
- Tool-use accuracy;
- Mean latency.

No benchmark score is hard-coded into the repository.

6.2 Optional RAGAS Evaluation

Optional RAGAS model-judged evaluation adds:

- Faithfulness;
- Answer relevance;
- Context precision;
- Context recall.

RAGAS scoring requires evaluator model credentials and remains distinct from the directly observed metrics. LM Evaluation Harness is not part of the runtime but can be used independently for future general-capability regression testing.

6.3 MLflow Tracking

Agent-run tracking is optional and disabled by default. Each `/ask` run can record the question, retrieved chunks and scores, generated answer, citations, action trace, tool usage, latency, and relevant configuration.

Evaluation metrics and case-level artifacts can also be recorded for comparison across runs.

The tracking configuration variables and example values provided in the source README are:

Setting	Source README example	Role
`SOURIVA_MLFLOW_ENABLED`	`true`	Enables optional agent-run tracking
`SOURIVA_MLFLOW_TRACKING_URI`	`sqlite:///mlflow.db`	SQLite tracking database URI
`SOURIVA_MLFLOW_EXPERIMENT`	`baseline-rag`	Experiment identifier

6.4 Optional Model Adaptation

AgentSouriva includes experimental entry points for adapting a model to the desired agent behavior. These training components are separate from the core runtime and are not required for document retrieval and grounded answering.

Successful supervised traces contain the following fields:

```
question
retrieved_context
desired_action
grounded_answer
citations
```

A validated example trace is located at `data/sft_example.jsonl`. Supervised fine-tuning is based on these successful traces. The LoRA approach can be extended to QLoRA, in which supported systems load the base model in 4-bit mode.

The preference-training interface defines an explicit reward signal around supported answers, citation correctness, appropriate tool use, and abstention. Unsupported claims, invented citations, and unnecessary tool calls are penalized.

Training and preference optimization remain optional, distinct from the core runtime.

7 Project Build Evolution

7.1 Project skeleton and configuration

Source commit: `9ba1ed1`

7.1.1 Change

Added the installable `souriva` package, project metadata, dependency groups, ignore rules, and environment-driven runtime settings.

Overall Project Structure and Configuration. AgentSouriva uses three complementary files to organize its Python project. The `pyproject.toml` file defines project metadata, build and installation requirements, dependencies, and development tool configurations. The `config.py` file manages runtime settings, including storage, retrieval parameters, model selection, and experiment tracking. Finally, `__init__.py` identifies the `souriva` directory as a Python package, executes package initialization code when imported, and exposes package-level information such as `__version__`. In summary, `pyproject.toml` configures the software project, `config.py` configures the running application, and `__init__.py` defines the package's initialization and public interface.

Python Project Configuration (`pyproject.toml`). The `pyproject.toml` file is the standard configuration file for a Python project. It defines how the project is built and installed, specifies package metadata (e.g., name, version, and supported Python versions), and declares required and optional dependencies. For AgentSouriva, core dependencies support API serving, document processing, and retrieval, while optional dependency groups provide additional capabilities such as MCP integration, evaluation, training, and experiment tracking. The file also configures development tools, including `pytest` for testing and `ruff` for linting and formatting. By centralizing these settings, `pyproject.toml` simplifies installation, dependency management, and development.

Runtime Configuration (`config.py`). AgentSouriva centralizes its runtime configuration in a `Settings` class that inherits from Pydantic's `BaseSettings`. This separates configurable parameters from application logic, allowing behavior to be changed without modifying source code. Settings are loaded from environment variables prefixed with `SOURIVA_` or a local `.env` file, falling back to predefined defaults. Pydantic also validates and converts values according to their declared types, while `Literal` restricts the model backend to supported options.

The configuration covers storage locations, embedding dimensions, document chunking and overlap, retrieval limits and relevance thresholds, model backend and connection details, and MLflow experiment tracking. The `@property` decorator defines computed paths for the FAISS index and its associated metadata, ensuring that both remain consistent with the configured storage directory and index name.

Finally, the `get_settings()` function uses Python's `@lru_cache` decorator to instantiate the configuration only once and reuse it across subsequent calls. This avoids repeated environment parsing and provides a consistent configuration throughout the running process. Together, these mechanisms provide centralized, validated, environment-specific, and reusable runtime configuration.

The central settings object is the key configuration boundary:

```
class Settings(BaseSettings):
```

```
    model_config = SettingsConfigDict(env_prefix="SOURIVA_")
    data_dir: Path = Path("artifacts")
    model_backend: Literal["mock", "openai-compatible"] = "mock"
```

These declarations give every runtime component typed defaults while allowing each value to be overridden through a consistently prefixed environment variable.

7.1.2 Architecture

This establishes the configuration boundary shared by the API, retrieval, model, tracking, and evaluation components. Runtime artifacts are kept outside source control.

7.1.3 Design notes

The default backend is a local mock and the default index lives under `artifacts/`. Optional dependency groups keep MLflow, MCP, evaluation, and training stacks out of the core installation. Python is capped below 3.14 because several binary ML dependencies do not yet publish compatible wheels.

7.1.4 Verification

Ran `python -m compileall souriva` and inspected the staged diff. Compilation succeeded.

7.1.5 Result

AgentSouriva now has a package boundary and one centralized, environment-overridable configuration object.

7.2 Model adapter and local backend

Source commit: `5474d04`

7.2.1 Change

Added the model protocol, shared evidence and reply values, and a deterministic extractive mock backend.

The protocol captures the narrow contract implemented by every model backend:

```
class ModelAdapter(Protocol):
    def answer(
        self, question: str, evidence: Sequence[Evidence]
    ) -> ModelReply: ...
```

Agent code therefore depends only on an `answer` operation that consumes evidence and returns normalized text plus citation identifiers, not on a particular inference library.

7.2.2 Architecture

The adapter separates agent behavior from model serving. Evidence enters through a small typed contract and model replies identify the chunk IDs they cite.

7.2.3 Design notes

The mock selects at most two chunks and extracts one sentence from each. It is deliberately deterministic so ingestion, retrieval, grounding, and API behavior can be exercised without a running language model.

7.2.4 Verification

Ran `python -m compileall souriva tests`. The modules compiled successfully. The test runner is added as a development dependency and tests will run after dependencies are installed.

7.2.5 Result

The project can generate a predictable evidence-linked reply through the same interface a GPT-OSS-20B adapter will implement.

7.3 GPT-OSS-compatible model adapter

Source commit: `d26e836`

7.3.1 Change

Added a small OpenAI-compatible HTTP adapter and backend selector.

The adapter serializes the grounded prompt into the standard chat-completions request shape:

```
context = "\n\n".join(
    f"[{item.id}] {item.text}" for item in evidence
)
payload = {
    "model": self.model,
    "temperature": 0,
    "messages": [
        {"role": "system", "content": system},
        {"role": "user", "content":
         f"QUESTION:\n{question}\n\nEVIDENCE:\n{context}"},
    ],
}
```

This small translation layer lets a locally served GPT-OSS model use the same `ModelAdapter` interface as the deterministic mock backend.

7.3.2 Architecture

GPT-OSS-20B serving is isolated behind the same `ModelAdapter` boundary as the mock. The adapter sends retrieved context to a chat-completions endpoint and returns a normalized reply.

7.3.3 Design notes

The prompt marks retrieved text as untrusted data and requires evidence-only answers. Returned citation identifiers are intersected with retrieved chunk IDs so fabricated citations cannot enter the application response. A standard-library HTTP client avoids another core dependency.

7.3.4 Verification

Ran module compilation and checked the diff. Factory unit tests cover both configured backends and invalid configuration; full test execution awaits development dependency installation.

7.3.5 Result

A local GPT-OSS-20B server can later replace the mock through configuration without changes to agent logic.

7.4 Document loader

Source commit: `b7bd2a2`

7.4.1 Change

Added source-aware loading for PDF, Markdown, and UTF-8 text documents.

The format branch preserves page information for PDFs and a document location for text files:

```
if suffix == ".pdf":
    return [
        DocumentPart(text=text, source=path.name,
                     location=f"page {number}")
        for number, page in enumerate(PdfReader(path).pages, start=1)
        if (text := (page.extract_text() or "").strip())
    ]
```

Each extracted part carries its filename and location immediately, so later chunking and citation stages do not need to reconstruct provenance.

7.4.2 Architecture

The loader is the first RAG pipeline stage. It normalizes files into `DocumentPart` values while preserving filenames and page locations for downstream citations.

7.4.3 Design notes

PDF extraction uses one part per non-empty page. Text formats remain one part at this stage so chunking stays independent of file I/O. Unsupported extensions fail explicitly instead of being guessed.

7.4.4 Verification

Compiled the package and tests, checked the diff, and added unit coverage for text, Markdown, and unsupported types. PDF extraction is exercised when the optional fixture is available in later integration testing.

7.4.5 Result

AgentSouriva can normalize all three required document formats into a common metadata-preserving representation.

7.5 Metadata-preserving document chunking

Source commit: `98e802b`

7.5.1 Change

Added bounded, overlapping text chunking with stable content-derived identifiers.

Each emitted chunk receives location metadata and a deterministic digest:

```
location = f"{part.location}, chunk {number}"
digest = hashlib.sha256(
    f"{part.source}:{location}:{text}".encode()
).hexdigest()[:12]
chunks.append(Chunk(id=f"chunk-{digest}", text=text,
                    source=part.source, location=location))
```

Hashing the source, location, and normalized text makes the identifier repeatable for equivalent content while the overlap loop preserves context between neighboring chunks.

7.5.2 Architecture

Chunking converts loaded document parts into the retrieval unit used by embeddings, FAISS, citations, and chunk lookup.

7.5.3 Design notes

Chunks prefer whitespace boundaries and retain controlled overlap to preserve context across edges. IDs are stable hashes of source, location, and text, making citations reproducible across equivalent ingestions without a global counter.

7.5.4 Verification

Compiled the modules, checked the diff, and added cases for overlap, metadata, stable IDs, and invalid sizing.

7.5.5 Result

Loaded documents can now be split into deterministic, citation-ready chunks.

7.6 Local embedding backend

Source commit: `584e85c`

7.6.1 Change

Added deterministic normalized feature-hashing embeddings.

The embedding loop maps each token to a signed dimension and normalizes the finished vector:

```
for token in TOKEN_PATTERN.findall(text.lower()):
    digest = hashlib.blake2b(token.encode(), digest_size=8).digest()
    value = int.from_bytes(digest)
```

```
    vector[value % self.dimensions] += 1.0 if value & 1 else -1.0
norm = np.linalg.norm(vector)
if norm:
    vector /= norm
```

The fixed hash function makes vectors reproducible without downloading a model, and normalization makes inner product usable as cosine-style similarity.

7.6.2 Architecture

This backend supplies vectors to the FAISS index while keeping initial operation independent of network access and model downloads.

7.6.3 Design notes

Token hashes select dimensions and signs, then vectors are L2-normalized for inner-product similarity. This baseline favors lexical overlap; the narrow `embed` interface can later be implemented by a learned embedding model without changing index behavior.

7.6.4 Verification

Compiled source and tests and checked the diff. Unit cases cover determinism, shape, normalization, relative similarity, and empty batches.

7.6.5 Result

AgentSouriva can produce repeatable local vectors suitable for FAISS cosine-style search.

7.7 Persistent FAISS index

Source commit: `11737e5`

7.7.1 Change

Added exact inner-product indexing, metadata persistence, index restoration, and chunk lookup.

Replacing the corpus rebuilds the vector index and persists it with its metadata sidecar:

```
def replace(self, chunks: Iterable[Chunk]) -> int:
    self.chunks = list(chunks)
    self.index = faiss.IndexFlatIP(self.embeddings.dimensions)
    if self.chunks:
        vectors = self.embeddings.embed(c.text for c in self.chunks)
        self.index.add(vectors)
    self.save()
    return self.count
```

Recreating `IndexFlatIP` makes replacement semantics explicit, while `save` writes both the FAISS data and the ordered chunk records needed to interpret its rows.

7.7.2 Architecture

`FaissStore` owns the vector index and its ordered chunk sidecar. The sidecar retains citation fields that FAISS itself does not store.

7.7.3 Design notes

Re-ingestion atomically replaces the logical corpus for predictable local behavior. Load validates both vector dimensions and row counts, preventing silent mismatches between the binary index and JSON metadata.

7.7.4 Verification

Compiled source and tests and checked the diff. Added persistence, lookup, and missing-index test cases. Runtime execution requires the declared NumPy and FAISS dependencies.

7.7.5 Result

Chunk vectors and citation metadata can now be saved locally and restored across process restarts.

7.8 FAISS retrieval

Source commit: `a6703bc`

7.8.1 Change

Added ranked vector search, score thresholding, and an ingestion pipeline function that joins loading, chunking, embedding, and index replacement.

The search path converts FAISS positions back into source-aware evidence and applies the relevance floor:

```
scores, positions = self.index.search(query_vector, limit)
for score, position in zip(scores[0], positions[0]):
    if position < 0 or float(score) < min_score:
        continue
    chunk = self.chunks[int(position)]
    results.append(Evidence(id=chunk.id, text=chunk.text,
                            source=chunk.source,
                            location=chunk.location,
                            score=round(float(score), 6)))
```

FAISS supplies scores and row positions; the sidecar supplies the text and citation metadata returned to callers.

7.8.2 Architecture

The retriever converts FAISS positions back into evidence objects carrying source, location, score, text, and stable citation ID.

7.8.3 Design notes

Search uses normalized embeddings with inner product, equivalent to cosine similarity. Empty queries and empty indexes return no evidence. A configurable minimum score provides the first explicit control for evidence adequacy.

7.8.4 Verification

Compiled all modules, checked the diff, and added retrieval cases for ranking, metadata, thresholding, and end-to-end text ingestion.

7.8.5 Result

AgentSouriva now has a complete local path from supported files to ranked, citation-ready evidence.

7.9 Grounding and abstention policy

Source commit: `08e292a`

7.9.1 Change

Added explicit post-generation checks for retrieval strength, valid citations, and model abstention.

The fail-closed decision keeps only sufficiently relevant evidence and rejects missing or unknown citations:

```
eligible = {item.id: item for item in evidence
            if item.score >= min_relevance}
if not eligible:
    return GroundedAnswer(ABSTENTION, (), True,
                          "insufficient_retrieval_score")
if not cited_ids or any(item_id not in eligible
                        for item_id in cited_ids):
    return GroundedAnswer(ABSTENTION, (), True,
                          "missing_or_invalid_citations")
```

This makes retrieval quality and citation membership enforceable application rules rather than relying only on the model prompt.

7.9.2 Architecture

The alignment module sits between model output and external responses. It converts a model reply into either a cited grounded answer or a standardized abstention with a machine-readable reason.

7.9.3 Design notes

Only evidence above the configured threshold is eligible. Citation IDs are deduplicated and resolved from that allow-list; replies without citations or with any unknown citation are rejected. This makes citation behavior inspectable rather than relying only on prompt compliance.

7.9.4 Verification

Compiled source and tests and checked the diff. Unit cases cover accepted citations, weak retrieval, invented citations, and missing citations.

7.9.5 Result

Unsupported output cannot pass through as an answer, and callers can distinguish why the system abstained.

7.10 Grounded agent flow

Source commit: `f71e9a1`

7.10.1 Change

Added the single-pass document agent and explicit action trace.

The coordinating method takes one retrieval-to-grounding path and records its outcome:

```
evidence = self.retriever.search(
    question, self.top_k, self.min_relevance
)
if not evidence:
    answer = GroundedAnswer(ABSTENTION, (), True,
                            "no_relevant_evidence")
    return AgentResult(answer, (),
                       (Action.RETRIEVE, Action.ABSTAIN))
reply = self.model.answer(question, evidence)
answer = ground_reply(reply, evidence, self.min_relevance)
```

The early return avoids generation when retrieval fails; otherwise the model reply must pass the grounding policy before becoming an agent result.

7.10.2 Architecture

The agent coordinates retrieval, model generation, and alignment without owning their implementation. Its trace uses the four conceptual actions `RETRIEVE`, `USE_TOOL`, `ANSWER`, and `ABSTAIN`.

7.10.3 Design notes

Document questions take the shortest path: one retrieval, at most one model call, then a grounding decision. No evidence causes an immediate abstention, avoiding unnecessary model work and reducing the chance of unsupported answers.

7.10.4 Verification

Compiled modules, checked the diff, and added isolated tests for the successful grounded path and pre-generation abstention path.

7.10.5 Result

The core can now coordinate retrieval and generation into a validated, traceable answer.

7.11 Restricted calculator

Source commit: `1d88d7a`

7.11.1 Change

Added a restricted arithmetic calculator for approved tool execution.

The recursive evaluator accepts only numeric constants and explicitly listed arithmetic nodes:

```
if isinstance(node, ast.Constant) and type(node.value) in (int, float):
    return float(node.value)
if isinstance(node, ast.BinOp) and type(node.op) in _BINARY:
    left, right = evaluate(node.left), evaluate(node.right)
    return _BINARY[type(node.op)](left, right)
raise ValueError("Only numeric arithmetic is allowed")
```

Names, calls, and attributes never match an allowed branch, so they are rejected instead of being evaluated as Python code.

7.11.2 Architecture

The calculator is transport-independent so the agent and MCP server can share identical behavior.

7.11.3 Design notes

The calculator walks a small arithmetic AST and rejects names, calls, attributes, oversized exponents, and extreme results.

7.11.4 Verification

Compiled source and tests and checked the diff. Cases cover normal arithmetic and attempted code execution.

7.11.5 Result

Approved arithmetic has a small, testable implementation ready for agent and MCP exposure.

7.12 MCP document and calculator tools

Source commit: `7c1be2f`

7.12.1 Change

Added the approved document tool surface and exposed document search, chunk lookup, and arithmetic through a FastMCP server.

The MCP layer registers narrow functions and delegates their work to `DocumentTools`:

```
@mcp.tool()
def search_documents(query: str, top_k: int = 4) -> list[dict]:
    return tools.search_documents(query, top_k=max(1, min(top_k, 20)))
```

```
@mcp.tool()
def get_document_chunk(chunk_id: str) -> dict | None:
    return tools.get_document_chunk(chunk_id)
```

The decorators expose only these approved operations, while delegation keeps retrieval behavior independent of the MCP transport.

7.12.2 Architecture

`DocumentTools` contains transport-independent behavior. `mcp_server` only constructs the configured store and maps approved methods to MCP, keeping protocol concerns out of retrieval and agent modules.

7.12.3 Design notes

The server restores the same persisted index used by the API. Search result counts are bounded to protect local resources, and no generic function dispatcher is exposed, so clients can invoke only the three registered tools.

7.12.4 Verification

Compiled all modules and checked the diff. Core tests cover search serialization, chunk lookup, missing IDs, arithmetic, and rejection of executable syntax.

7.12.5 Result

MCP clients can search the local corpus, inspect cited chunks, and perform controlled calculations over stdio.

7.13 Agent calculator routing

Source commit: `29056ba`

7.13.1 Change

Added an explicit calculator route to the document agent and completed behavior for all four agent actions.

An explicit prefix selects the calculator path before document retrieval begins:

```
def ask(self, question: str) -> AgentResult:
    if question.lower().startswith("calculate:"):
        expression = question.partition(":")[2].strip()
        return self._calculate(expression)
    evidence = self.retriever.search(
        question, self.top_k, self.min_relevance
    )
```

This deterministic branch prevents arithmetic requests from reaching retrieval or the language model and confines execution to the injected approved calculator.

7.13.2 Architecture

Questions prefixed with `calculate:` take a bounded tool path. The calculator is injected as a callable, keeping the agent unaware of MCP transport and limiting tool access to an approved function.

7.13.3 Design notes

Tool requests do not enter retrieval or the model. Expected parsing and arithmetic failures become a standardized abstention, while successful calculations are returned with a `USE_TOOL`, `ANSWER` trace and no document citations.

7.13.4 Verification

Compiled source and tests, checked the diff, and added cases for successful and rejected calculator requests.

7.13.5 Result

The agent now executes retrieval, approved tool use, answers, and abstentions through short deterministic paths.

7.14 FastAPI runtime and health endpoint

Source commit: `b12701e`

7.14.1 Change

Added centralized runtime construction and a FastAPI application with a health endpoint.

The health route reads readiness information from the single composed runtime:

```
app = FastAPI(title="AgentSouriva", version="0.1.0")
runtime = create_runtime()

@app.get("/health")
def health() -> dict:
    return {"status": "ok",
            "documents_ready": runtime.store.count > 0,
            "chunks": runtime.store.count,
            "model_backend": runtime.settings.model_backend}
```

Constructing the runtime once gives every request the same store, tools, model, and agent while the route reports state without invoking the model.

7.14.2 Architecture

The runtime wires configuration, embeddings, persisted FAISS state, tools, model adapter, and agent in one place. The API imports that composition root rather than constructing components per request.

7.14.3 Design notes

Startup restores an existing local index when present. Health reports readiness-relevant state without making model calls or exposing paths and credentials.

7.14.4 Verification

Compiled all modules, checked the diff, and added an isolated API test for an empty temporary data directory.

7.14.5 Result

`uvicorn app:app` can expose process health and confirm the selected backend and indexed chunk count.

7.15 Document ingestion API

Source commit: `fd96f13`

7.15.1 Change

Added validated request and response schemas and `POST /documents` for local file ingestion.

The endpoint passes validated paths and configured chunking limits into the shared ingestion pipeline:

```
@app.post("/documents", response_model=IngestResponse)
def ingest_documents(request: DocumentsRequest) -> IngestResponse:
    count = ingest_paths(runtime.store, request.paths,
                         chunk_size=runtime.settings.chunk_size,
                         overlap=runtime.settings.chunk_overlap)
    return IngestResponse(chunks=count,
                          sources=[p.name for p in request.paths])
```

FastAPI validates the request schema, and the thin route replaces the runtime corpus before returning the resulting count and source names.

7.15.2 Architecture

The endpoint is a thin adapter over the RAG ingestion pipeline and shared runtime store. It accepts server-local paths for PDF, Markdown, and text files and replaces the active corpus.

7.15.3 Design notes

Batch size is bounded at the schema. Expected file and format errors become HTTP 400 responses while unexpected failures remain visible. Responses report the resulting chunk count and submitted source names.

7.15.4 Verification

Compiled the code, checked the diff, and added API cases for successful text ingestion and a missing path.

7.15.5 Result

Clients can build the local FAISS corpus through a typed HTTP request.

7.16 Retrieval API

Source commit: `3bf0fef`

7.16.1 Change

Added typed evidence schemas and `POST /retrieve` for direct corpus search.

The route applies request bounds and the runtime relevance threshold to the same store used by the agent:

```
@app.post("/retrieve", response_model=RetrieveResponse)
def retrieve(request: RetrieveRequest) -> RetrieveResponse:
    evidence = runtime.store.search(
        request.query,
        top_k=request.top_k,
        min_score=runtime.settings.min_relevance,
    )
    return RetrieveResponse(evidence=[item.__dict__ for item in evidence])
```

The response model serializes complete evidence records, letting clients inspect ranking and citations without triggering answer generation.

7.16.2 Architecture

The endpoint exposes the same configured FAISS retriever used by the agent and MCP tools. Results preserve the stable chunk ID, text, source, location, and measured similarity score.

7.16.3 Design notes

Query length and result count are bounded by request validation. The global relevance threshold is applied consistently so direct retrieval and agent decisions observe the same evidence floor.

7.16.4 Verification

Compiled source and tests, checked the diff, and extended the ingestion integration test to restore and query the persisted corpus. Added request validation coverage for invalid `top_k`.

7.16.5 Result

Clients can inspect ranked evidence independently of answer generation.

7.17 Grounded question API

Source commit: `de904b3`

7.17.1 Change

Added `POST /ask` with typed answers, citations, abstention details, and action traces.

The endpoint exposes the agent result without reimplementing its retrieval or grounding logic:

```
@app.post("/ask", response_model=AskResponse)
def ask(request: AskRequest) -> AskResponse:
    result = runtime.agent.ask(request.question)
    return AskResponse(
        answer=result.answer.text,
        citations=[c.__dict__ for c in result.answer.citations],
        abstained=result.answer.abstained,
        reason=result.answer.reason,
        actions=[action.value for action in result.actions],
    )
```

Mapping the normalized agent result into a typed response preserves citations, policy decisions, and the action trace for HTTP clients.

7.17.2 Architecture

The endpoint delegates to the shared `DocumentAgent`, exposing alignment results without duplicating retrieval or grounding decisions in HTTP code.

7.17.3 Design notes

Responses keep citations structured for clients while the answer text also retains inline chunk references. Abstentions use HTTP 200 because they are successful, policy-compliant outcomes rather than transport failures.

7.17.4 Verification

Compiled the project, checked the diff, and extended API integration coverage for a cited document answer and an empty-corpus abstention.

7.17.5 Result

The complete ingest, retrieve, and grounded-answer flow is available through FastAPI.

7.18 MLflow agent-run tracking

Source commit: `ee865aa`

7.18.1 Change

Added centralized, opt-in MLflow logging around question requests.

The request boundary measures the whole agent call and submits the completed trace to the optional tracker:

```
started = perf_counter()
result = runtime.agent.ask(request.question)
```

```
latency_ms = (perf_counter() - started) * 1000
log_agent_run(request.question, result, latency_ms,
              runtime.settings)
```

Tracking after completion captures latency and final behavior in one place; `log_agent_run` returns immediately when MLflow is disabled.

7.18.2 Architecture

Tracking receives the completed agent result rather than instrumenting retrieval, models, or alignment separately. This keeps experiment concerns out of core behavior while retaining the full trace.

7.18.3 Design notes

Each run records configuration, latency, abstention, evidence and scores, response, citations, actions, and tool use. Tracking defaults off so the core install remains light; enabling it without the dependency produces a direct installation message.

7.18.4 Verification

Compiled the code, checked the diff, and added a test confirming disabled tracking has no side effects or MLflow import requirement.

7.18.5 Result

Setting `SOURIVA_MLFLOW_ENABLED=true` records query traces in the configured `baseline-rag` experiment.

7.19 Direct RAG evaluation metrics

Source commit: `1dacf66`

7.19.1 Change

Added a compact evaluation runner for retrieval, citation, unsupported-answer, abstention, tool-use, and latency metrics.

Each observed run is reduced to directly measurable Boolean outcomes before aggregation:

```
retrieval_hit = not case.expected_sources or bool(
    set(case.expected_sources) & retrieved_sources
)
rows.append({
    "retrieval_hit": retrieval_hit,
    "citation_correct": cited_ids <= retrieved_ids,
    "unsupported_answer":
        not result.answer.abstained and not cited_ids,
    "abstention_correct":
        result.answer.abstained is not case.answerable,
    "latency_ms": latency_ms,
})
```

These fields compare the returned evidence, citations, and policy outcome with each case definition, producing transparent metrics without an evaluator model.

7.19.2 Architecture

Evaluation operates on the public agent boundary and retains per-case answers, contexts, references, and measurements for optional RAGAS processing.

7.19.3 Design notes

Metrics are calculated only from observed runs. Retrieval Recall@k is represented by expected-source hits for the initial dataset. Citation correctness verifies that every returned citation was among retrieved evidence, while unsupported-answer rate detects uncited non-abstentions.

7.19.4 Verification

Compiled source and tests, checked the diff, and added deterministic cases covering an answerable question, a correct abstention, all aggregate direct metrics, and empty-dataset validation.

7.19.5 Result

AgentSouriva can produce transparent baseline quality and latency measurements without requiring an evaluator model.

7.20 RAGAS evaluation runner

Source commit: `5b0d127`

7.20.1 Change

Added a four-case evaluation dataset, two source documents, a direct evaluation CLI, and optional RAGAS execution for faithfulness, answer relevance, context precision, and context recall.

The command always computes direct metrics and runs model-judged RAGAS metrics only when requested:

```
direct, rows = evaluate_cases(runtime.agent, cases)
output = {"direct": direct}
if args.ragas:
    output["ragas"] = run_ragas(rows)
print(json.dumps(output, indent=2))
```

This conditional keeps the default benchmark local and reproducible while preserving an explicit path to evaluator-model scoring.

7.20.2 Architecture

The runner builds an isolated evaluation index, exercises the real runtime, then transforms observed answers and contexts into the RAGAS dataset schema. Model-judged metrics run only with `--ragas`.

7.20.3 Design notes

RAGAS is pinned to the 0.2 API line used by the runner. The default command reports only locally measured direct metrics; RAGAS requires its evaluator model credentials and never substitutes invented scores.

7.20.4 Verification

Compiled the runner and checked the diff. Full execution is deferred until the declared FAISS and NumPy dependencies are installed.

7.20.5 Result

`python -m benchmarks.ragas_eval` runs the reproducible local suite, while `--ragas` adds the four requested evaluator-model metrics.

7.21 MLflow evaluation tracking

Source commit: `5458832`

7.21.1 Change

Added MLflow recording for aggregate evaluation metrics and case-level outputs, with a CLI flag on the evaluation runner.

One MLflow run stores summary measurements alongside the full case records:

```
with mlflow.start_run(run_name="evaluation"):
    mlflow.log_params({
        "model_backend": settings.model_backend,
        "top_k": settings.retrieval_top_k,
        "cases": len(rows),
    })
    mlflow.log_metrics(metrics)
    mlflow.log_dict({"cases": rows}, "evaluation.json")
```

The aggregate values support run comparison, while the JSON artifact retains the evidence needed to audit each score.

7.21.2 Architecture

Evaluation tracking remains centralized beside agent-run tracking. The benchmark runner passes only measured metrics and observed case records after execution.

7.21.3 Design notes

Direct and RAGAS metric names are kept distinct, configuration is recorded as parameters, and detailed cases are stored as one JSON artifact. Tracking remains opt-in through `--mlflow`.

7.21.4 Verification

Compiled the project, checked the diff, and extended disabled-tracking coverage to ensure evaluation does not require MLflow unless requested.

7.21.5 Result

Evaluation runs can be compared in the configured MLflow experiment without adding instrumentation to core RAG code.

7.22 LoRA and QLoRA SFT entry point

Source commit: `19ed186`

7.22.1 Change

Added an optional supervised fine-tuning command for successful grounded traces, plus a validated example record.

The training entry point constructs a parameter-efficient adapter and passes it to TRL:

```
lora = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                  target_modules="all-linear")
config = SFTConfig(output_dir=str(args.output),
                   dataset_text_field="text", num_train_epochs=1)
trainer = SFTTrainer(model=model, args=config,
                     train_dataset=dataset, tokenizer=tokenizer,
                     peft_config=lora)
trainer.train()
```

Only the LoRA parameters are trained over validated trace text, keeping adaptation separate from the serving-time runtime.

7.22.2 Architecture

Training stays outside the runtime package. Trace formatting defines the integration contract between collected agent runs and TRL, while PEFT supplies a LoRA adapter over the configured base model.

7.22.3 Design notes

The command trains LoRA by default and enables 4-bit QLoRA explicitly. Required trace fields preserve the question, retrieved context, chosen action, grounded answer, and citations. Imports are lazy so runtime users do not load training libraries.

7.22.4 Verification

Compiled the entry point, checked the diff, and added unit coverage for formatted action/citation content and incomplete trace rejection. A full GPT-OSS-20B training run was not executed because it requires separate model weights and accelerator resources.

7.22.5 Result

`python -m training.sft data.jsonl` provides a focused path from successful traces to a saved LoRA adapter.

7.23 Preference optimization reward contract

Source commit: `63cd6be`

7.23.1 Change

Added a trainer-neutral reward function and preference-pair record for future RL or direct preference optimization.

The reward makes the desired grounding behavior and its penalties explicit:

```
reward = 0.0
reward += 1.5 if signals.supported_answer else 0.0
reward += 1.0 if signals.citations_correct else 0.0
reward += 0.75 if signals.tool_use_correct else 0.0
reward += 1.0 if signals.abstention_correct else 0.0
reward -= 2.0 if signals.unsupported_claim else 0.0
reward -= 1.5 if signals.invented_citation else 0.0
```

Keeping the weights in a plain function makes the initial preference contract inspectable and independent of any particular trainer.

7.23.2 Architecture

The module converts measurable grounding outcomes into a scalar reward outside the runtime agent. A downstream TRL trainer can consume the normalized prompt, chosen response, and rejected response without changing evaluation logic.

7.23.3 Design notes

Supported answers, valid citations, correct tools, and correct abstentions receive positive weight. Unsupported claims and invented citations receive the strongest penalties, with a smaller penalty for unnecessary tool calls. The explicit weights are a baseline to calibrate from observed preference data.

7.23.4 Verification

Compiled the project, checked the diff, and added a test confirming supported behavior scores above an unsupported answer and produces the expected preference record. No optimization result is claimed or fabricated.

7.23.5 Result

The project has a concrete data and reward boundary for later RL or preference optimization without adding it to the initial runtime.

7.24 Installation entry points

Source commit: `9cd644b`

7.24.1 Change

Added pip requirements entry points for runtime and development/evaluation environments and allowed newer Python versions when dependency wheels are available.

The two requirements files select optional groups from the shared package metadata:

```
# requirements.txt
-e .[mcp,tracking]

# requirements-dev.txt
-e .[dev,evaluation,mcp,tracking]
```

Both environments therefore inherit one set of version constraints from `pyproject.toml`; the development entry point simply enables additional testing and evaluation tools.

7.24.2 Architecture

Core package metadata remains the dependency source of truth. Requirements files select the relevant optional groups rather than duplicating version constraints.

7.24.3 Design notes

The standard requirements include MCP and MLflow functionality. Development requirements additionally include tests and RAGAS evaluation; accelerator-heavy training dependencies remain an explicit separate extra.

7.24.4 Verification

Checked the configuration diff and created a local virtual environment. Dependency installation and the complete suite are the next verification step.

7.24.5 Result

Users have direct installation commands for the working service and for repository development.

7.25 Dependency-backed verification fixes

Source commit: `4055d6f`

7.25.1 Change

Included the API, benchmark, and training entry points in package discovery, excluded successful tool-only responses from the unsupported document-answer metric, normalized imports with Ruff, and used native async ASGI request handling and tests.

The principal packaging and metric corrections are visible in these focused lines:

```
[tool.setuptools]
py-modules = ["app"]
[tool.setuptools.packages.find]
include = ["souriva*", "benchmarks*", "training*"]
```

```
unsupported_answer = (
    not result.answer.abstained
    and Action.USE_TOOL not in result.actions
    and not cited_ids
)
```

Package discovery now installs every executable module, while the metric excludes successful calculator results because tool-only answers intentionally have no document citations. Converting the API handlers and ASGI tests to `async` completed the Python 3.14 compatibility fix.

7.25.2 Architecture

Installed command modules and `app:app` now resolve outside a source checkout. Evaluation distinguishes grounded document claims from approved tool results, which do not require document citations.

7.25.3 Design notes

The initial benchmark exposed the metric distinction: a correct calculator response had no document citation by design. It remains covered by tool-use accuracy and is no longer treated as an unsupported factual document answer. Native async ASGI requests avoid a Python 3.14 incompatibility in the deprecated synchronous test-client bridge.

7.25.4 Verification

Installed core, development, and MCP dependencies into `.venv`. After the initial failures exposed these issues, ran `pytest -q` (37 passed), `ruff check .`, `ruff format --check .`, the direct benchmark, and imports for the API, MCP, and training entry points. All completed successfully. The benchmark measured 1.0 retrieval hit, citation correctness, abstention accuracy, and tool-use accuracy; unsupported-answer rate was 0.0. Latency is environment-dependent and printed by the run.

7.25.5 Result

Packaged entry points and direct metrics now reflect their intended runtime semantics.

7.26 Complete model citation validation

Source commit: `aa8973d`

7.26.1 Change

Changed model-response citation parsing to preserve every bracketed identifier for grounding validation.

The parser now forwards all emitted identifiers instead of silently filtering unknown ones:

```
def parse_citation_ids(text: str) -> tuple[str, ...]:
    return tuple(re.findall(r"\[([^\[\]]+)\]", text))

return ModelReply(text=text,
                  citation_ids=parse_citation_ids(text))
```

This preserves fabricated identifiers long enough for the alignment layer to detect that they are outside the retrieved evidence allow-list and abstain.

7.26.2 Architecture

The model adapter extracts citation syntax, while the alignment module remains responsible for checking each identifier against eligible retrieved evidence.

7.26.3 Design notes

Previously the adapter discarded unknown IDs before alignment, so a response containing both a real and fabricated citation could retain the fabricated text while passing the structured check. Passing all emitted IDs forward makes the existing fail-closed policy complete.

7.26.4 Verification

Added a regression test for mixed known and unknown IDs. Ran the full test suite and lint after the change; results are recorded in the final documentation verification.

7.26.5 Result

Any bracketed citation outside the retrieved evidence allow-list now causes the response to abstain.

7.27 Optional stack compatibility bounds

Source commit: `56a702e`

7.27.1 Change

Bound the RAGAS 0.2 LangChain dependencies below 0.4 and changed local MLflow 3 storage from the deprecated file store to SQLite.

The compatibility fix combines explicit evaluator bounds with a supported local tracking URI:

```
evaluation = [
  "ragas>=0.2,<0.3",
  "langchain>=0.2,<0.4",
  "langchain-community>=0.2,<0.4",
  "langchain-openai>=0.1,<0.4",
]
mlflow_tracking_uri: str = "sqlite:///mlflow.db"
```

The upper bounds retain the module layout expected by RAGAS 0.2, and SQLite replaces the file-store backend rejected by MLflow 3.

7.27.2 Architecture

These changes affect only optional tracking and evaluator-model dependencies. Core API, retrieval, MCP, and mock-model behavior remain unchanged.

7.27.3 Design notes

MLflow 3 places filesystem tracking in maintenance mode and rejects it by default, while its SQLite backend remains supported. RAGAS 0.2 allows newer LangChain packages in its metadata even though its imports require the pre-1.0 module layout. Explicit LangChain bounds make the evaluator extra reproducible.

7.27.4 Verification

The first optional-stack smoke run reproduced both failures: MLflow rejected `file:./mlruns`, and RAGAS failed importing a module removed from LangChain 1.x. A trial MLflow 2 pin was rejected because its PyArrow constraint has no Python 3.14 wheel. Reinstalled MLflow 3 with the bounded evaluator dependencies. RAGAS imports succeeded, direct evaluation created the SQLite experiment and logged its run, all 38 tests passed, and Ruff checks passed. Evaluator-model RAGAS scoring was not run because no evaluator credential was configured.

7.27.5 Result

Tracking uses `sqlite:///mlflow.db` by default, and the RAGAS adapter imports against a compatible dependency set.